



Court Case Backlog and Justice Delay Observatory

A scalable judicial analytics pipeline for monitoring court delays, backlog dynamics, and resolution efficiency

Event-driven judicial data engineering pipeline combining incremental ingestion, layered storage architecture, KPI computation, and interactive analytics for court backlog monitoring.

Asia Panizza (255334) - asia.panizza@studenti.unitn.it,
Yasmin El Morady(267977) - yasmin.elmorady@studenti.unitn.it,
Henri Vasserot(264384) - henri.vasserot@studenti.unitn.it

Team, Roles, And Collaboration

Who worked on what, how the team collaborated, and each member's background.

Content

Asia Panizza — MSc in Economics, BSc in International Relations

Yasmin El Morady — BSc in Psychology

Henri Vasserot — BSc in Mathematics & Philosophy

Asia Panizza

- Exploratory data analysis
- KPI validation
- Documentation
- Bronze/Silver/Gold architecture

Yasmin El Morady

- Dashboard design
- Visual analytics
- Streamlit integration

Henri Vasserot

- System architecture
- Incremental ingestion pipeline
- Metrics computation

Workflow and collaboration

- Distributed workload across analytics, engineering, and visualization tasks
- GitHub-based collaborative development
- Shared Google documents and repository coordination
- Online and presential meetings

Project At A Glance

A compact summary of the project.

- **Problem** : Judicial backlog and court resolution delays
- **Target user/stakeholder** : Policy makers, court administrators, legal researchers
- **Input data** : Event-based judicial records from the CourtListener API
- **Main result** : Interactive judicial analytics dashboard and scalable monitoring pipeline
- **Pipeline summary** : CourtListener API → Bronze → Silver → Gold → Streamlit Dashboard
- **Key metric** : Case resolution time between filing and resolution

Problem And Motivation

Why this project matters and what it is trying to solve.

- **Problem statement** : Judicial systems face growing backlogs and delays due to imbalance between incoming and resolved cases.
- **Why it matters** : Long judicial delays reduce fairness, transparency, and efficiency of justice systems.
- **Goal or success metric** : Build a scalable data-driven system for monitoring court delays, backlog dynamics, and judicial performance indicators.
- **Scope** : Multi-court and multi-year analysis of event-based judicial records.

Example use case

A policy maker identifies courts with excessive delays and reallocates resources.

Target audience

Policy makers, court administrators, researchers.

Main assumption

Judicial processes can be modeled as event-based data with measurable delays.

Why This Is A Big Data Problem

Explain the scale or complexity that motivates the architecture.

Volume : Large-scale judicial event records across courts and multiple years.

Variety : Heterogeneous judicial data including timestamps, jurisdictions, procedural events, and case states.

Velocity or refresh rate : Continuous updates as new judicial events and case records are generated.

Why the chosen architecture helps : Bronze/Silver/Gold architecture enables scalable ingestion, incremental updates, reliable storage, and efficient analytical processing.

Approximate data size

~5M+ judicial records in the database supported by runtime ingestion

Growth expectation

Continuous dataset expansion through incremental historical ingestion and new case updates

Key bottleneck

Efficient processing and aggregation of large temporal event-based datasets

Data Sources, Quality, And Scope

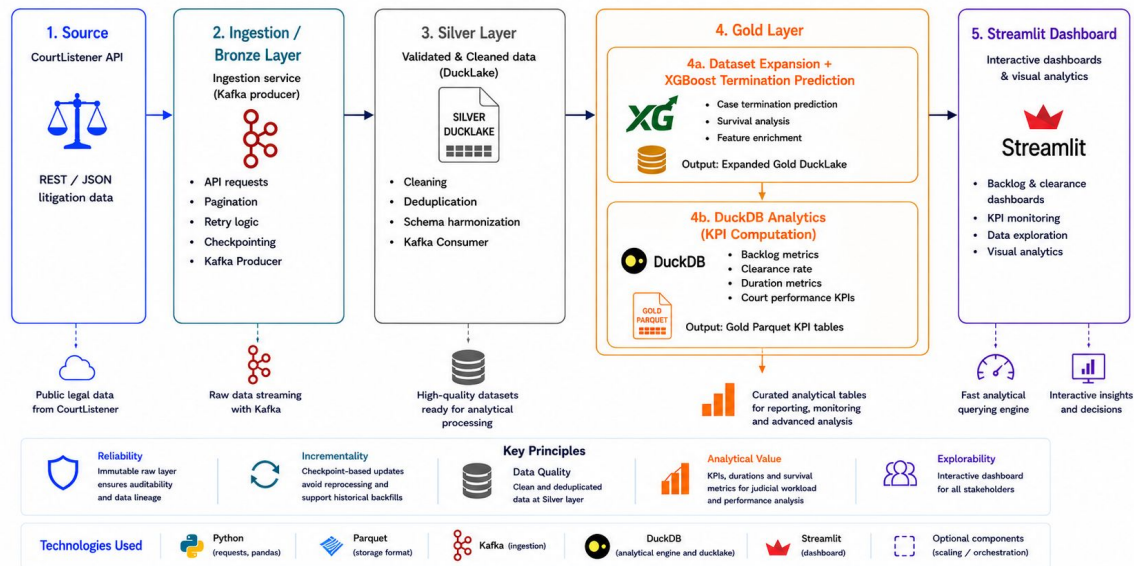
Summarize the data used in the project.

Data sources	Coverage and format	Quality issues or constraints	Preprocessing steps
CourtListener public litigation API	Coverage period: 1934–2026 (strongly biased in the period 2023-2026)	Incomplete judicial coverage	Paginated API ingestion
REST / JSON event-based judicial records	~5M+ judicial records in database	Strong imbalance between active and resolved cases	Temporal normalization
Multi-court procedural and temporal case data	~ discretionary runtime ingestion	Sparse termination observations	Schema harmonization
	JSON ingestion → parquet analytical layers	Right-censoring effects on duration metrics	Deduplication and cleaning
	Multi-year event-based litigation dataset	Data volume	Bronze → Silver → Gold transformation workflow using Kafka and DuckDB (ducklakes)

Architecture Overview

Judicial Analytics Pipeline – Medallion Architecture

From CourtListener data to analytical insights



Data Flow, Runtime View

How data moves from ingestion to output.

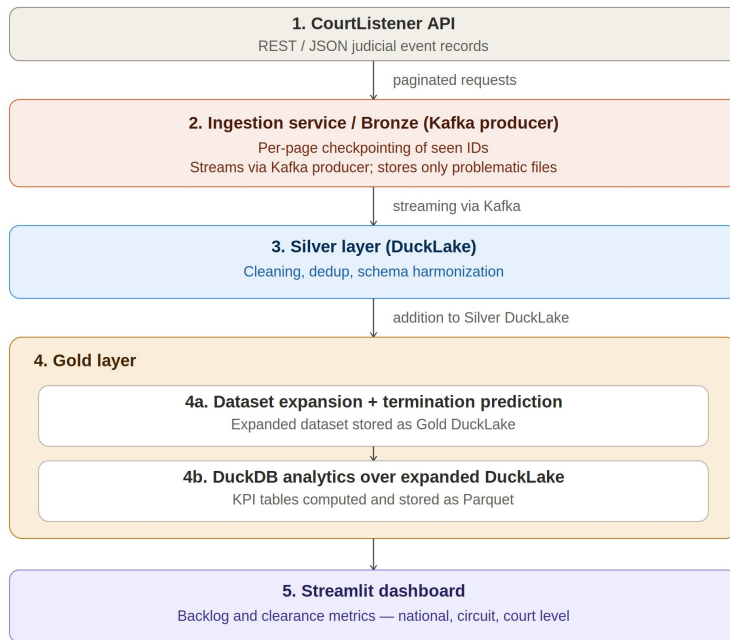
Runtime pipeline

Incremental ingestion using
Kafka

Bronze → Silver → Gold
workflow

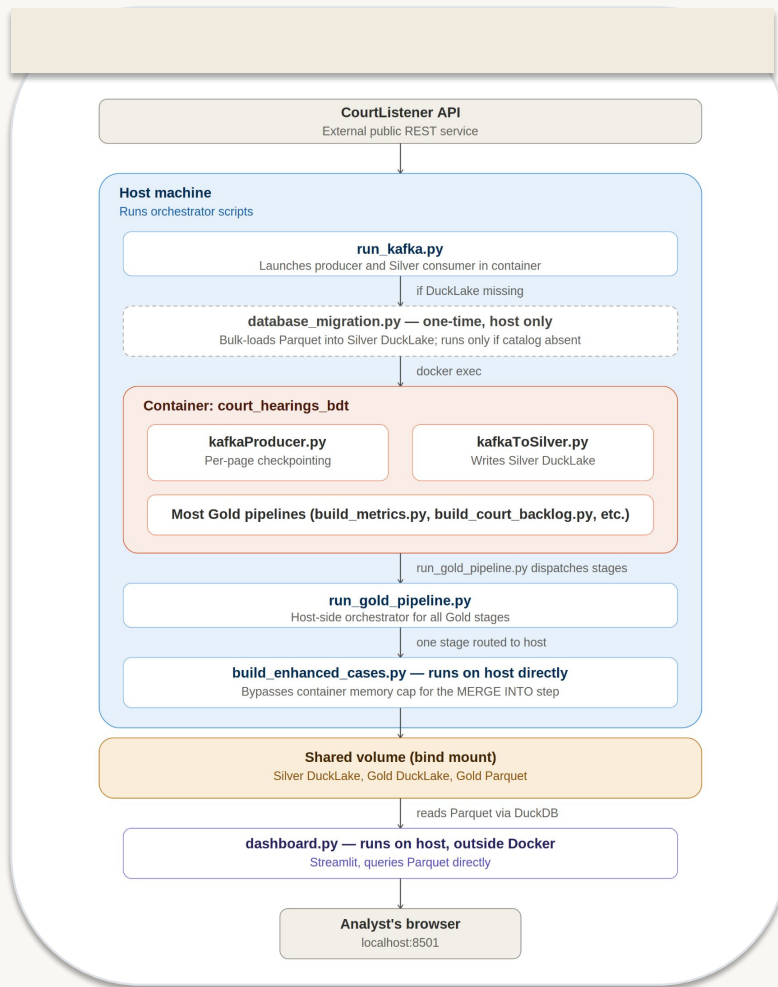
Docker-supported
execution

Analytical KPI generation



Deployment view

Host fallback avoids
container memory limits



Design Decisions And Trade-offs

Key technology choices behind the project.

Technology

Purpose

Python

Data ingestion & processing

Parquet

Analytical storage

DuckDB

Fast OLAP querying, storage in ducklakes

Streamlit

Interactive dashboard

CourtListener API

Judicial data source

Kafka

Streaming ingestion

Main trade-offs

- Local pipeline over distributed infrastructure, but first draft of potential distribution
- Batch incremental processing instead of real-time streaming
- Simplicity prioritized over large-scale orchestration

Reproducibility And Repository

Where the code lives and how to run the project.

Repository & structure

Modular repository organization
Separate ingestion, processing, Bronze, Silver, Gold, and dashboard layers
GitHub-based version control

Prerequisites

Python 3.12 environment
Required dependencies from [requirements.txt](#)
Docker
CourtListener API access, register [here](#) and follow the instructions on the README file in the repository
Download files from [here](#) and place them in correct folder

How to run it

ingestion and processing: `python run_kafka.py`; `python run_gold_pipeline.py`
dashboard: `python -m streamlit run "./dashboard.py"`

Inputs and outputs

Input: CourtListener judicial event records
Output: parquet analytical datasets and dashboard KPIs

Output Paths

silver / data /
gold / data /
gold / metrics /

Results: Functional Evidence

Main outputs and task-level results.

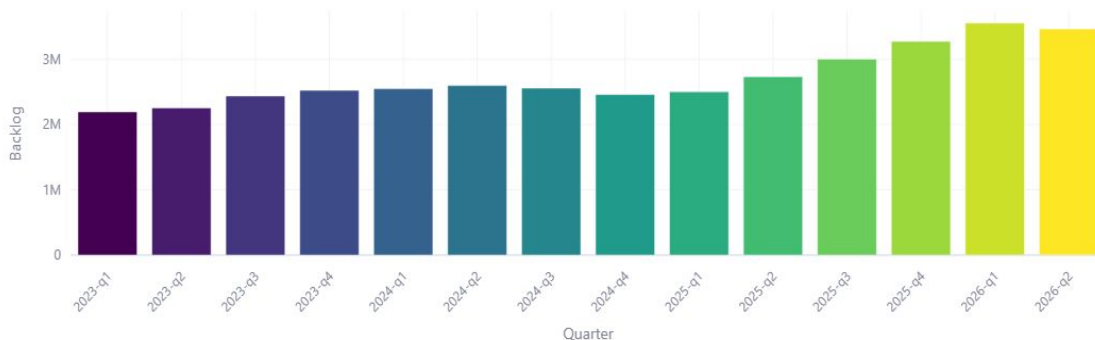
Metric : Judicial backlog evolution

Baseline : Incoming cases exceed resolved cases

Output example : Temporal backlog accumulation across years

Takeaway : Persistent inflow > outflow drives long-term judicial congestion

Nationwide Backlog — by quarter



Persistent inflow > outflow generates sustained backlog accumulation

Results: System And Performance Evidence

Runtime: run as a self-contained incremental batch with per-page checkpointing of seen IDs and delivery-confirmed

Throughput : Interactive analytical querying enabled by DuckDB and parquet storage

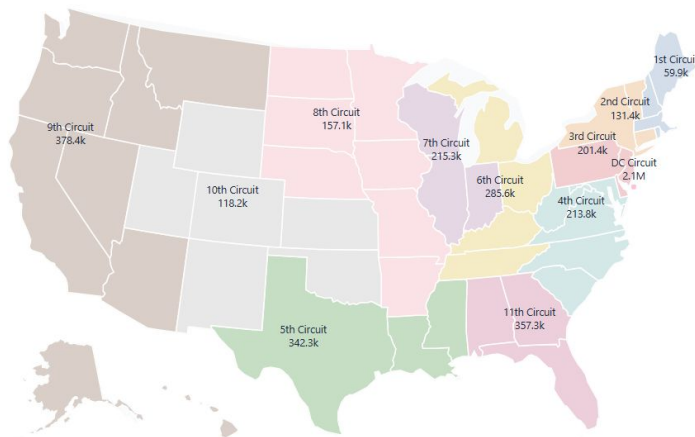
Memory or cost : Columnar parquet storage reduces analytical overhead and supports efficient querying

Storage efficiency : data compressed into lightweight parquet analytical layers

Scalability : Scalability: seen ids deduplication prevents reprocessing of records, so ingestion cost scales with the size of each incremental delta, not the size of the historical dataset.

Bottleneck : Strong active/resolved imbalance and temporal aggregation complexity

Backlog by circuit — 2026-q2



Limitations, Lessons, And Next Steps

Wrap up the project with an honest reflection.

Current limitations :

Incomplete judicial coverage from the public API (using backlog from 2020 onwards and cases terminated from 2023 onwards)

Strong imbalance between active and resolved cases

Main lesson learned :

Incremental Medallion architectures provide reliable analytical workflows for temporal judicial data

Next-step :

Automated scheduled ingestion

Extended analytical KPIs

Larger-scale judicial monitoring

Scaling concern :

Larger historical ingestion may eventually require distributed orchestration

Open question :

How can judicial congestion dynamics be modeled predictively over time?

References, Acknowledgments, And GenAI Disclosure

Sources, external help, and acknowledgments.

Datasets and references :

CourtListener public litigation API
Public judicial event records and metadata

Libraries, API, and tools :

Python / pandas / DuckDB / Streamlit / parquet / Docker / Kafka

Acknowledgments :

University of Trento — Big Data Technologies course
CourtListener open judicial data initiative

GenAI usage :

Generative AI tools were used for code assistance, debugging support, documentation refinement, and presentation structuring, as well as for court processing.

Dataset link

original:

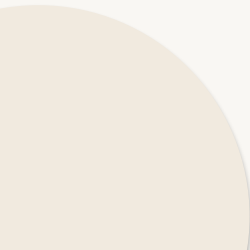
<https://www.courtlistener.com/api/>

pre-processed:

https://drive.google.com/drive/folders/1-pogtrR4fofkjftPa86cw_hEXPVQ0G4G?usp=sharing

Contribution credits

Data engineering and architecture
Dashboard and analytical visualization
KPI design and backlog analytics



Thank you for your attention